VS - KVS

→ Verteilte Systeme:

→ Persistenz: Dauerhafte, permanente Speicherung

→ Konsistenz: Parallele Prozesse haben gleiche Sicht auf die Daten

• Dateisysteme:

→ Persistent gespeicherte Datenmenge

→ Hierarchischer Namensraum

→ Verzeichnis: Datei mit einer Zuordnung von Namen zu internen Datei-IDs

→ gemeinsame, gleichzeitige Nutzung mit Zugriffskontrolle

↳ Zugriffsmethoden ↳ Sequentieller Zugriff

→ Metadaten

→ Datei: Größe, etc

→ Verzeichnis: Inhalt, etc

→ Gerät: Superblock

• Verteiltes Dateisystem

Ein Verteiltes Dateisystem unterstützt die gemeinsame Nutzung von Dateien in einem Netz. Es emuliert die Funktionalität eines nicht verteilten Dateisystems und ermöglicht die Nutzung entfernter Dateien genau wie Dateien im lokalen Dateisystem.

VS - KVB - 8

→ Anforderungen an verteilte Dateisysteme

- Zugriffstransparenz:
  - → Gleiche Zugriffsoperationen wie für lokale Dateien, d.h. Verteilung bleibt Klienten verborgen
- Namenstransparenz:
  - → Einheitlicher Namensraum
- Mobilitätstransparenz:
  - → Dateiverschiebungen sind für Client-Programme transparent
- Leistungstransparenz:
  - → Gleiche Leistung innerhalb eines Lastbereiches
- Fehlertransparenz:
  - → Nach Ausfall von C/S kann Betrieb wieder korrekt weiterlaufen
- Nebenläufigkeitstransparenz:
  - → Gleichzeitige Zugriffe auf eine Datei durch verschiedene Clients stören sich nicht
  - → Nebenläufigkeitskontrolle
- Dateireplikation
  - → Dateidienste können Replikten von Dateien anlegen
  - → Konsistenzprobleme müssen gelöst werden
  - → Vorteile: Fehlertoleranz / Erhöhung der Skalierbarkeit
- Heterogenität:
  - → Heterogene Clients müssen unterstützt werden
- Sicherheit

VS-KÜB

→ Semantik verteilter Dateizugriffe:

1) One Copy Semantik (Unix)
   - Alle Änderungen werden sofort sichtbar
   - es existieren keine Kopien der Datei
   - eventuell sehr große Latenzzeiten
   - entspricht strikter Konsistenz

2) Upload/Download Modell
   - Sitzungs-Semantik
     - → Klient meldet Dateizugriff an und erhält Kopie,
     - → Modifikation an Datei werden erst nach close sichtbar
     - → Server-initiierte Validierung: Server teilt Client mit, falls sich Datei geändert hat.
     - → Verlustproblem, falls mehrere Clients gleichzeitig Datei ändern
       - ↳ KSG: Merge

3) Read-Only Semantik
   - Idee: Datei wird einmal geschrieben und bleibt dann unveränderlich
   - Schreibzugriffe durch Client erfolgen atomar und erzeugen neue Datei
   - Erleichtert die Replikation, aber die Verlustproblematik ist wieder da

4) Transaktionssemantik
   - Zugriffe auf eine Datei werden in Transaktionen gebündelt
   - ein Transaktionsmonitor garantiert ACID Eigenschaften
   - erfordert Verwaltungsaufwand

VS - KV3

→ Zustandslose und zustandsbehaftete Dateiserver:

- Zustandsinformationen:
  - → Welche Dateien werden durch welchen Client geöffnet, und stellt Positionszeiger einer Datei?
  - → Wer verwaltet Zustandsinformationen: Client / Server?
- Vorteile zustandsloser Dateiserver:
  - → Benötigt keinen Speicher für Zustand
  - → Operationen zum Öffnen / Schließen von Dateien nicht notwendig
  - → Fehlertoleranz einfach realisierbar, da keine Zustandsinformationen zwischen den Servern synchronisiert werden muss
  - → Beim Absturz eines Clients entstehen für Server keine Probleme, da keine veraltete Information vorhanden
- Vorteile zustandsbehafteter Dateiserver:
  - → Nachrichten kleiner, da keine Zustandsdaten mit jedem Zugriff übertragen werden
  - → Schreib- & Lesezugriffe sind effizienter, da z.B. die Positionszeiger einer mit Zugriff befindlichen Datei bereits am richtigen Platz stehen
  - → Precaching auf Server möglich, da alle Dateien bekannt sind, die sich im Zugriff befinden
  - → Dateisperren können vom Server unterstützt werden
  - → POSIX ist zustandsbehaftet

VS - RVS

→ Caching & Replikation:

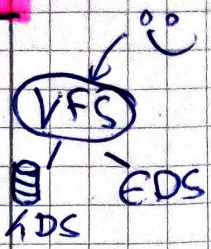
- Caching durch den Server:
  - → Pufferung nur im Hauptspeicher des Servers
  - → Eliminiert nur Disktransferzeiten
  - → Aus Klientensicht nur eine einzige Kopie
  - → Klient schreibt immer auf Originaldatei
  - → Keine Konsistenzprobleme
  - → Netzlatenz ist immer noch vorhanden
- Replikation von Dateien:
  - → Verteilung der Last auf mehrere Server
  - → Absicherung gegenüber Datenverlust und Serverausfällen
  - → Strikte Replikationskontrolle:
    - Lock / unlock entsprechend der Dateioperation
    - Invalidate für ungültige gewordene Kopien
    - Update zur sofortigen Aktualisierung
  - → Schwache Replikationskontrolle
    - Schreiboperationen auf primäre Kopie
    - Verzögerte Ausbreitung der Änderung
  - → Abstimmungsverfahren (Voting):
    - zwischen den Servern bezüglich verschiedener Versionen

VS - K V3

→ Client Caching:

- → Eliminiert Plattenzugriffe und Nachrichtentransport
- → Verschiedene Konsistenzprotokolle möglich
  - ↳ zentrale Vergabe der Zugriffsrechte beim Server
  - ↳ verschiedene Durchschreibstrategien
    - → sofortiges Durchschreiben auf Server
    - → verzögertes Durchschreiben auf Server
    - → zurückschreiben beim Close der Datei
- → Unterbringung des Puffers
  - ↳ im Adressraum des Prozesses
  - ↳ im BS-Kernel
  - ↳ auf Festplatte

VS - KVB



Network File System (NFS)

- NFS-Server Modul befindet sich im Kernel
- Benutzer-Programme können Datei-Operationen gleichermaßen für lokale und entfernte Dateien verwenden.
- Integration erfolgt über Virtual File System
- Zugriffstransparenz:
  - Clients können Zugriffe auf lokale und entfernte Dateien in gleicher Weise durchführen
- Ortstransparenz:
  - Dateisysteme oder Teile davon werden zum Export angeboten
  - Client gibt an, an welcher Stelle im lokalen Verzeichnis das importierte Dateisystem eingehängt wird (mount point)
- Fehlertransparenz:
- Nebenläufigkeitstransparenz:

Dateigruppen:

Dateigruppe: Menge von Dateien auf einem Server

- Ein Dateisystem ist eine Dateigruppe
- Ein NFS exportiertes Verzeichnis ist eine Dateigruppe

- Dateigruppen in VS müssen eindeutig identifizierbar sein
- Die virtuelle Integration einer Dateigruppe in eine andere nennt man <u>mounten</u>.

VS-KVB

Mount-Dienst:

- Bei NFS übernimmt der Mount-Dienst Prozess des Mountens entfernter Dateisysteme
- Auf Server gibt es Datei mit Namen lokaler Dateisysteme, die gemountet werden können
- Mount-Punkt: Zeigt den Ort des Mounts im lokalen Dateisystem
- Automounter verwaltet Tabelle mit Mount-Punkten und Verweisen auf NFS-Server
- Automounter übernimmt transparent das Mounten eines entfernten Verzeichnisses beim ersten Zugriff auf leeren Mount-Punkt

NFS-Client-Caching:

- Vor jedem Zugriff erfolgt eine Gültigkeitsprüfung
- Hierzu haben alle Blöcke im Cache zwei Zeitstempel
  - T_c: Zeit, zu der Cache-Eintrag zuletzt geprüft wurde
  - T_m: Zeit, zu der Block zuletzt auf Server geändert
- Cache-Eintrag ist zu einer Zeit t gültig:
  - Wenn $T - T_c < t$ (t heißt Aktualisierungsintervall) oder
  - wenn der auf Client gespeicherte Wert für T_m mit dem Wert von T_m auf Server übereinstimmt, d.h.
    $(T - T_c < t)$ OR $(T_{m_{client}} == T_{m_{server}})$
  - + Kompromiss Konsistenz & Effizienz
  - Beim Schreibzugriff wird entsprechender Block im Cache als dirty markiert

VS - KVB

⇝ Direct Attached Storage (DAS):

- → An einzelnen Rechner oder Server angeschlossene Festplatten; Punkt-zu-Punkt Verbindung
- → Schnittstellen: SCSI
- → Andere Rechner haben nur über Zugangsrechner zugreifen
- → Begrenzte Kapazität
- → Zugriff: blockweise
- → Dateisystem wird durch Zugangsrechner verwaltet

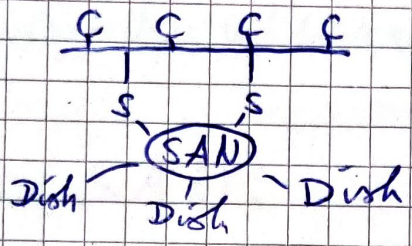
⇝ Network Attached Storage (NAS):

- → Speicher, der an ein Netz angeschlossen ist
- → Autonomer Serverdienst, der dem über das LAN angeschlossenen Clients Dateisysteme zur Verfügung stellt.
- → Festplatten sind an keinen Server gebunden
- → Zugriff: Datei-basiert: NFS
- → Große Datenmenge, geringer Energieverbrauch
- → Leicht zu administrieren

VS - KVB

→ Storage Area Network (SAN):

- → Netzwerk zu schnellen Anbindung von Festplattensystemen an Server-Systeme; many-to-many Verbindung
- → Mehrere Speichersysteme können an mehrere Server angebunden werden



- → Große Datenmengen
- → Datenzugriff ist blockweise, das Dateisystem wird durch zugreifenden Rechner verwaltet
- → Anbindung: Hochleistungsnetzwerk direkt an I/O Bus der Rechner
- → Einfache zentrale Verwaltung
- → Backup außerhalb vom SAN
- → Virtualisierung von Plattenbetriebssystemen
  - ↳ Der verteilt vorhandene Massenspeicher wird virtuell wie eine einzige Festplatte behandelt
  - ↳ Einzelnen Servern werden auf virtueller Festplatte Partitionen zugewiesen, die die Server wie eigene einzige Festplatte einbinden.
  - ↳ Vorteil: Vorhandener Speicherplatz kann effektiver genutzt und zentral verwaltet werden
  - ↳ Im laufenden Betrieb kann Größe des zugewiesenen Speicherbereichs geändert werden.

VS - KVB

⇝ Storage Area Network:

→ SAN stellt sich SAN als ein kohärenter Massen-Speicher zur Verfügung, obwohl SAN aus vielen kleinen Speichern bestehen kann

→ SAN ist für schnelle Datenübertragung zwischen einzelnen Speichern und Servern optimiert.

⇝ NAS vs SAN:

- Datenzugriff: → SAN und DAS blockweise
  → NAS dateibasiert
- Backup: → Übernimmt bei SAN und DAS der Server
  → Bei NAS muss dies anderer Rechner übernehmen

⇝ Cloud Speichersysteme

→ Speicherung der Daten auf Speichersystemen bei Cloud-Anbietern

→ Zugang über Internet

→ Synchronisierung der Dateien und Ordner zwischen lokaler Festplatte und Cloud-Speicher

→ Einfache Versionskontrolle

VS-KVB

- Server-Konsolidierung

Die Zusammenlegung von virtuellen Servern auf wenige physikalische Server nennt man Server-Konsolidierung

↳ bessere HW Auslastung

↳ Vorteile: • Kostensenkung der HW
- Vereinfachte Administration
- Vereinfachte Bereitstellung neuer Server
- Unterstützung von Altanwendungen

- Virtualisierung

Nachbildung von Hardware-Objekten durch funktional gleichwertige Objekte von selben Typ mit Hilfe einer Softwareschicht.

- Virtuelle Maschine

Eine virtuelle Maschine bildet die Rechnerarchitektur eines real in Hardware existierenden Rechners nach, z.B. Hardware A auf Hardware B.

VS-KVB

→ Physischer Rechner
- → HW: CPU, Speicher-, E/A Module, Ressourcen oft nicht ausgelastet
- → SW: Eng an HW gekoppelt, Ein BS abstrahiert von HW

→ Virtualisierung: Hardware Level
- → Abstraktion der HW
  - → Virtuelle CPU, Speicher, E/A-Module
  - → Umfasst Zustand des BS und Anwendungen
- → Virtualisierungsschicht
  - → Spezielle Softwareschicht entkoppelt HW und BS
  - → Physische HW wird multiplexed
  - → Virtualisierungsschicht verwaltet physische Ressourcen. Diese werden jedem BS vorgetäuscht, als ob sie tatsächlich vorhanden wären

VS-KVB

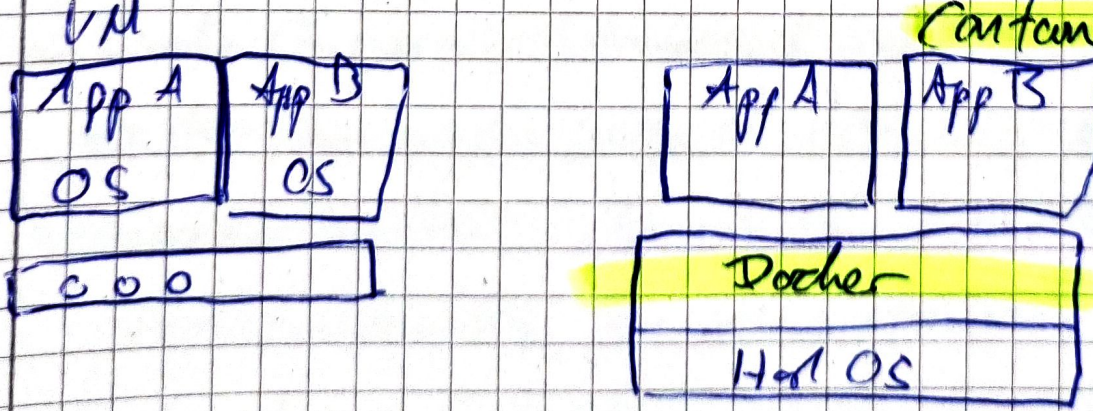
⇝ Vorteile der Virtualisierung:

(+) Reduktion der Anzahl der Rechner
(+) Erhöhung der Auslastung der Rechner
(+) Energieeinsparung durch optimale Rechnerauslastung
(+) Unabhängigkeit von HW
(+) Migration von Servern im laufenden Betrieb
(+) Vereinfachte Softwareinstallation
(+) Erhöhung der Verfügbarkeit

⇝ Nachteile der Virtualisierung:

(−) Bei Virtualisierung auf HW-Level benötigt jede VM eigenes Gastbetriebssystem mit
↳ Folge: Größeres Image, erhöhter CPU- und Arbeitsspeicherverbrauch

⇝ Alternative: Anwendungen laufen in Container, die teilen sich ein BS



VS-KVS

→ Managed Application

Eine managed application ist eine Anwendung, welche in einer kontrollierten Umgebung durch Anwender installiert und ausgeführt wird. Dazu wird Anwendung samt der genutzten Bibliotheken und weiteren Abhängigkeiten in eine Einheit (Image) gespeichert.

→ App wird von Infrastruktur getrennt
→ Image wird durch Container überführt
→ Container: → Sind vergleichbar mit VMs
→ werden streng von einander isoliert ausgeführt.

eg.: Docker
→ Docker führt managed applications in container aus
→ Docker container ist im Prinzip ein Prozess
→ Docker Host: Stellt Umgebung bereit, in der Container ablaufen
→ Docker Client: Kommunikation des Anwenders mit dem Daemon
→ Docker Container: Container ist isolierte, sichere Applikationsplattform
→ Container werden gestartet, gestoppt, verschoben, gelöscht
→ Jeder Container wird aus dem Docker Image gebildet und beinhaltet alles, was eine lauffähige App benötigt
→ Docker Image: Vorlage nach der ein Container erstellt wird
→ Docker Registry: Docker Images werden in Registry vorgehalten und können von dort in Container geladen werden

VS-KVB

- **Cloud Computing**

  Unter Cloud-Computing versteht man die massive Bündelung von Rechenressourcen an zentrale Stelle durch einen Anbieter. Ressourcen können über ein Netzwerk durch Anwender genutzt werden.

- **Infrastructure as a Service (IaaS)**

  IaaS stellt IT-Ressourcen wie Speicher, Rechenleistung oder Netzwerkkapazitäten zur Verfügung. Anwender haben dabei die Kontrolle über BS und Anwendungen. Sie sind für den Betrieb der Software selbst verantwortlich.

↳ Vorteile: → RechenInstanzen können je nach Anforderung beliebig erweitert oder verkleinert werden

→ Benutzer hat vollen Zugriff auf Instanzen, dabei ist er aber der BS-Ebene selbst verantwortlich

VS-KVS

- Platform as a Service (PaaS)

PaaS stellt Programmiermodelle und Entwicklerwerkzeuge bereit, um Anwendung zu erstellen und auszuführen. Anbieter stellen benötigte Ressourcen wie Rechenleistung, Speicher, Netzwerk, Load Balancing und Datenbanken automatisch beim Deployment der Applikation bereit. Diese skalieren in Abhängigkeit von den Anforderungen.

→ Ausrichtung auf die eigentlichen Verarbeitungseinheiten übernimmt Cloud
→ Benutzer hat keinen direkten Zugriff auf Rechenressourcen, er bekommt keine virtuellen Server

VS-KVB

- **Software as a Service (SaaS)**

Bei SaaS stellt Anbieter Anwendungen für Benutzer bereit. Anwender müssen sich weder um die technische Infrastruktur noch um Installation/Updates kümmern. Diese erfolgen zentral durch den Anbieter, sodass stets die aktuelle Version der Software zur Verfügung steht.

→ Cloud ist für Benutzer vollkommen transparent
→ Zugriff auf die Software meist über Webbrowser
→ Serviceanbieter übernimmt die komplette IT-Administration und weitere Dienstleistg

→ **Cloud-Computing:**

Vorteile: (+) von überall zugreifbar
(+) keine Datensicherg
(+) keine Softwarepflege

Voraussetzg: Überall Breitband
Netz-Verfügbarkeit

VS - KVB

- **Synchronisation**

Synchronisation bezeichnet das zeitliche Abgleichen von Vorgängen eines Uhren.

↳ Synchronisation sorgt dafür, dass Vorgänge
→ gleichzeitig oder
→ in einer bestimmten temporalen Reihenfolge geordnet ablaufen

↳ Synchronisierung: Inhaltliches Abgleichen von Replikaten

↳ eg: Prozesssynchronisation, Angleichen von Uhren, kausale Sortierung von Ereignissen

→ Korrektur der Uhrzeit:
→ Jeder Rechner besitzt eine Uhr, welche zum Zeitpunkt t Uhrzeit C(t) anzeigt
→ Prinzipien der Uhrenkorrektur
1) Sprünge von Uhrzeiten dürfen nicht vorkommen
2) Monotonie: Für alle $t_1 > t_2$ gilt $C(t_1) \geq C(t_2)$
→ Uhren haben oft unterschiedliche Geschwindigkeiten
→ Ideale Uhr: $C(t) = t$, d.h. $dC/dt = 1$
→ Schief: $S = dC/dt - 1$, Offset: $O = C(t) - t$
→ Eine Uhr arbeitet innerhalb ihrer Spezifikation, wenn es eine Konstante ρ (Driftrate) gibt, so dass jederzeit $|S| \leq \rho$

VS - KVS

→ Uhrensynchronisation:

→ Die Uhren der Rechner R_i sind

→ mit Uhren des Rechners S im Intervall I extern mit Genauigkeit D synchronisiert wenn $|S(t) - C_i(t)| < D$ für alle Rechner R_i und alle Zeiten t in I gilt.

→ untereinander im Intervall I intern mit Genauigkeit D synchronisiert wenn $|C_j(t) - C_i(t)| < D$ für alle Rechner R_i, R_j und alle Zeiten t in I gilt.

→ Sind die Uhren von Rechnern mit der Uhr eines Rechners S im Intervall I extern mit Genauigkeit D synchronisiert, so sind sie intern mit der Genauigkeit 2 D synchronisiert.

- Externe Synchronisation: Alle Rechner synchronisieren sich regelmäßig mit einem Server
- Interne Synchronisation: Alle Rechner synchronisieren sich regelmäßig untereinander, es gibt keinen zentralen Server.
- Ziel: Vor Überschreitung der maximal tolerierbaren Abweichung synchronisieren

VS - KVB

↝ Algorithmus von Cristian (Ext. Synchr.):

↳ Zeitserver auf UTC Basis

↳ Ziel: Externe Synchronisation mit Zeitserver

↳ Jeder Rechner R_i sendet regelmäßig Anfrage nach aktueller Uhrzeit an Zeitserver dieser schickt seine Zeit C_{UTC} zurück

→ t_0: Zeitpunkt zu dem Anfrage abgeschickt wird

→ t_3: Zeitpunkt zu dem Antwort ankommt

→ Θ: Zeit zur Verarbeitung der Anfrage (C_{UTC} wird zum Zeitpunkt $t_1 + \Theta$)

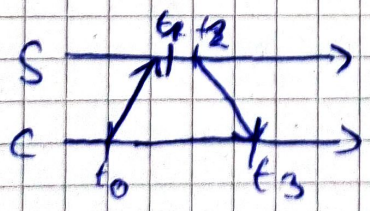
↳ Bearbeitungszeit $\Theta = t_2 - t_1$; Gesamtlaufzeit $\delta = (t_3 - t_0) - \Theta$

↳ Algorithmus basiert auf Annahme, dass Laufzeiten $t_1 - t_0$ und $t_3 - t_2$ gleich sind d.h. Korrekturzeit: $\delta / 2$

↳ Client empfängt Zeitstempel C_{UTC} setzt Zeit auf $C(t) = C_{UTC} + \frac{\delta}{2}$

↳ Ist Θ unbekannt, dann setze $\Theta = 0$

↳ Verbesserung: Führe viele Anfragen durch, diejenige die als schnellste zurückkommt wird als exakteste betrachtet



VS-K VB

→ Berkeley Algorithmus (Int. Synchro.):

→ Zeitserver Z aktiv, Ziel: Interne Synchronisation

→ Z schickt in regelmäßigen Abständen jedem Rechner seine Zeit

→ Jeder Rechner R_i antwortet mit seiner Abweichung A_i von der Zeit von Z

→ Z berechnet Durchschnitt $D = (A_1 + .. + A_n) / (n+1)$

→ Z schickt jedem Rechner R_i Korrekturzeit $D_i = D - A_i$

→ Alle Rechner korrigieren ihre Uhrzeit (auch Z selbst)

→ Z kann anstatt Durchschnitt auch andere Werte schicken

→ Verfeinerung: Z lässt bei Berechnung von D Ausreißer weg.

→ Kritik: Berkley und Cristian werden zentrale Komponenten und haben alle verbundenen Nachteile

(+) Bottleneck (+) Schlechtes Fehlertoleranzverhalten

Beim Berkeley Algorithmus kann prinzipiell jeder Rechner die Rolle des Zeitservers übernehmen, d.h. diese Funktion kann weitergegeben werden.

VS - KVB

→ Network Time Protocal: NTP (Exto Syndro)

- Ziele: → Internetweit sollen Clients gemäß UTC synchronisierbar sein
  → Dienst soll skalierbar und zuverlässig sein
  → Komponenten- oder Verbindungsausfälle und Über-
  tragungsverzögerungen müssen toleriert werden

- Realisierung:
  → Hierarchie von Servern verteilt über Internet
  → Primäre Server sind direkt mit UTC Zeitquelle verbunden
  → Sekundäre Server synchronisieren sich mit primären, etc.
  → Die einzelnen Level heißen Strata
  → Nachrichtenaustausch erfolgt über UDP port 123
  → Genauigkeit nimmt zu den Blättern ab.

VS-KVB

- **Kausalität**

Kausalität ist die Beziehung zwischen Ursache und Wirkung, sie betrifft die Abfolge von Ereignissen. Ein Ereignis A ist Ursache für Ereignis B, wenn B von A hervorgerufen oder beeinflusst wird.

- **Kausalrelation:**

Eine Relation → auf der Menge der Ereignisse in einem VS heißt Kausalrelation falls gilt:

→ Sind a und b Ereignisse im gleichen Prozess und tritt a vor b auf so gilt $a \rightarrow b$

→ Ist a ein Sendeereignis und b das zugehörige Empfangsereignis, dann gilt $a \longrightarrow b$.

→ $\longrightarrow$ ist transitive: $a \rightarrow b \wedge b \rightarrow c \Rightarrow a \rightarrow c$

Sind Ereignisse $a \wedge b$ unvergleichbar, so heißen sie nebenläufig $a \| b$ ($\neg a \rightarrow b \wedge \neg b \rightarrow a$).

VS KVS

- **Logische Uhr**

Eine logische Uhr ordnet Ereignissen in einem VS Zeitstempel zu, so dass die Kausalität der Ereignisse erkennbar ist: Wenn Ereignis e_1 ein Ereignis e_2 beeinflussen kann, dann muss e_1 einen kleineren Zeitstempel als e_2 haben

⇝ **Lamports Zeitstempel-Algorithmus**

→ Rechner R_i hat logische Uhr C_i ($C_i = 0$ beim Start)

→ Je nach Art des Ereignisses wird neuer Zeitstempel erzeugt

↳ Bei jedem internen Ereignis [redacted] inkrementiert R_i seine logische Uhr, d.h. C_i++

↳ Beim Senden einer Nachricht m inkrementiert der sendende Rechner R_i seine logische Uhr und hängt den neuen Wert von C_i als $C(m)$ an die Nachricht m an

↳ Beim Empfang einer Nachricht m mit angehängtem $C(m)$ aktualisiert der empfangende Rechner R_i seine logische Uhr gemäß:

$$C_i = \max(C_i, C(m)) + 1$$

- **Lamport Zeitstempel**

Der Lamport Zeitstempel-Algorithmus ordnet jedem Ereignis e einen numerischen Zeitstempel $C(e)$ so zu, so dass aus $a \to b$ folgt $C(a) < C(b)$

VS-K1/3

→ Kaufpol-Zeitstempel:

→ Mithilfe einer Erweiterung wird eine totale Ordnung aller Ereignisse in einem VS erreicht.

↳ Voraussetzung: Rechner haben eindeutige ID

↳ Zeitstempel ist Paar (Kaufpol-Zeit, Rechner-ID)

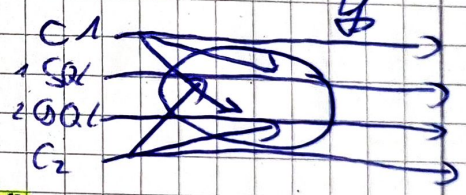
↳ $(\alpha_i, R_1) < (\alpha_j, R_2)$ genau dann wenn $\alpha_i < \alpha_j$ oder $\alpha_j = \alpha_j$ und $R_1 < R_2$

↳ Damit erhält jedes Ereignis ein eindeutigen Zeitstempel

→ Uhr kann überlaufen! ↯ :(

→ EG: Bankenhausinvariante

↳ Alle Operationen müssen auf jedem Rechner in der gleichen Reihenfolge durchgeführt werden!



↳ Vollständig sortierter Multicast:

→ Beim vollständig sortierten Multicast können Rechner einer Rechnergruppe Nachrichten an alle Mitglieder senden. Nachrichten werden bei allen Teilnehmern in der gleichen Reihenfolge ausgeliefert, d.h.

↳ liefert ein Rechner eine Nachricht m_1 aus, bevor er m_2 ausliefert, dann liefert jeder andere Rechner der m_2 ausliefert, ebenfalls m_1 vor m_2 aus.

Annahmen: 1) Nachrichten gehen nicht verloren

2) Nachrichten eines Rechners x an einen Rechner B kommen in gleicher Reihenfolge an, in der sie gesendet wurden

3) Anzahl der beteiligten Rechner ist allen bekannt

VS-KVS

⇝ Vollständig sortierter Multicast:

→ Alle MSGs enthalten einen erweiterten Lamport-Zeitstempel (mit Rechner ID) des Absende-Ereignis

→ MSGs werden immer an alle verschickt und dort gemäß ihrem Zeitstempel in Queue verwaltet

→ Empfänger sendet Empfangsbestätigung an alle, diese kommen auch in Queue

→ Rechner übergibt MSG zur Auswertung genau, dann wenn
1) sich MSG am Anfang der Queue befindet &
2) von jedem anderen Rechner eine MSG mit größerem Zeitstempel in Queue ist.

→ Nach Auslieferung wird MSG und alle zugehörigen Bestätigungs-nachrichten aus Queue entfernt.

VS - KVB

⇝ Vektor-Uhren:

→ Lamport-Uhren haben den Nachteil, dass aus $C(a) < C(b)$ nicht $a \to b$ gefolgert werden kann

→ Für Vektor-Uhren gilt: $a \to b \iff C(a) < C(b)$

→ Idee: → Zeitstempel ist nicht mehr einzelne Zahl sondern Vektor, dessen Komponenten die logischen Zeiten aller Rechner enthält

→ Mit jeder MSG werden die beim Sender bekannten Informationen über die Uhren aller Rechner ausgetauscht.

→ Umsetzung: Rechner R_i verwaltet Vektor V_i

Anzahl der Komponenten entspricht Anzahl der Rechner

1) Beim Start von Rechner R_i ist $V_i[k] = 0$ $\forall$ Rechner R_k
2) Bei jedem Ereignis von R_i (internes Ereignis, Senden, Empfangen) inkrementiert R_i seine Vektoruhr: $V_i[i] = V_i[i] + 1$
3) Beim Senden einer Nachricht m hängt R_i zusätzlich zu 2) den inkrementierten Wert von V_i als $V(m)$ an Nachricht m an
4) Beim Empfangen einer Nachricht m mit Wert $V(m)$ aktualisiert der empfangende Rechner R_j zusätzlich zu 2) alle Komponenten k seiner Vektoruhr in Form: $V_j[k] = \max(V_j[k], V(m)[k])$

VS - KVB

⇒ Vektor-Uhren:

→ Bedeutung der Komponenten von V_k:

1) $V_k[k]$ ist die Anzahl der Ereignisse, die bisher in Rechner R_k aufgetreten sind

2) Rechner R_k weiß, dass in Rechner R_j bisher mindestens $V_k[j]$ Ereignisse aufgetreten sind

2.1) In R_j sind möglicherweise mehr Ereignisse aufgetreten, aber davon ist noch keine Information bei R_k eingetroffen

→ Definition des Vergleichs von Vektor-Zeitstempeln:

- $C(a) = C(b) \iff \forall i$ gilt $C(a)[i] = C(b)[i]$
- $C(a) \leq C(b) \iff \forall i$ gilt $C(a)[i] \leq C(b)[i]$

→ Die zeitliche Beziehung zwischen a & b lässt sich aus Zeitstempeln lesen.

- $a \to b \iff C(a) < C(b)$
- $a \parallel b \iff (\neg [C(a) \leq C(b)] \wedge \neg [C(b) \leq C(a)])$

- **Kausale Ordnung**

Nachrichten in einem VS sind bei der Auslieferung kausal geordnet, falls Empfangsereignisse dieselbe Ordnung haben wie Sendeereignisse.

↳ Vektor Uhren

↳ send(m_1) → send(m_2) ⇒ receive(m_1) → receive(m_2)

VS - KVB

↝ Vereinfachte Vektor Uhren für Anwendung:

→ Vereinfachte Form des Vektor-Uhren genügt

1) $V_i[i]$ wird nur inkrementiert, wenn P_i eine NSG sendet, der Rest bleibt

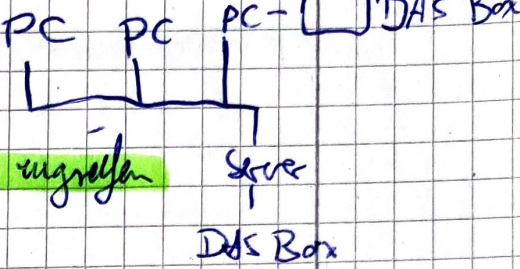
VS - KVB - ExS

(S-7)

a)

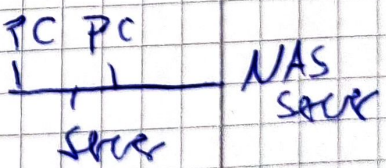
DAS (Direct Attached Storage)

- ↳ An einzelnen Rechnern oder Server angeschlossene Festplatten; Point to Point
- ↳ Serielle Schnittstelle
- ↳ Andere Rechner haben nur über Zugangsrechner zugriffen
- ↳ Begrenzte Kapazität
- ↳ Zugriff: blockweise
- ↳ Dateisystem wird durch Zugangsrechner verwaltet



NAS (Network Attached Storage)

- ↳ Speicher, der an ein Netzwerk angeschlossen ist
- ↳ Autonomer Serverdienst, der den über ein LAN angeschlossenen Clients Dateisystemen zur Verfügung stellt
- ↳ Festplatten sind an keinen Server gebunden
- ↳ Zugriff: Datei-basiert
- ↳ Große Datenmenge, geringer Energieverbrauch
- ↳ Leicht zu administrieren



Vektle S – KVB-Gr 5

(S1)

a)

SAN (Storage Area Network)

↳ Netzwerk zur schnellen Anbindung von Festplattensystemen an Server-Systemen many-to-many Verbindungen.

↳ Mehrere Speichersysteme können an mehrere Server angebunden werden

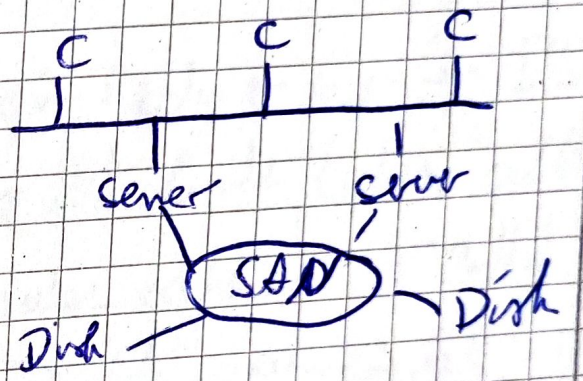
↳ Große Datenmenge

↳ Dateizugriff blockweise

↳ Einfache Zentralverwaltung

↳ Virtualisierung

↳ SAN stellt sich als ein Massenspeicher zur Verfügung



VS-KVB-6x S

(S-1)

c) Einige Zeitalgorithmen bilden aus den Uhrzeiten zweier Rechner das arithmetische Mittel und nehmen dieses Werk als neue Uhrzeit.
↳ §: Welt kann nun schlechter geändert werden :(

d) Betrachten sei das Verhalten zweier Rechner in einem VS. Beide haben Uhren, die 1000 pro Millisekunde ticken sollen. Eine davon hält davon, aber die andere tickt nur 999 Mal pro Millisekunde. Wie groß ist der Drift, wenn einmal pro Minute eine UTC-Aktualisierung eingeht?

1000 pro ms ; 999 pro ms
↓
60 000 000 pro min ; 56 940 000 pro min

160 000 000 - 56 940 000 | = 3 060 000 = p
↳ 60 ms/min

VS-KVB-Ex 5

(S-D)

	Cristian	Berkeley
Ziel	Extern	Intern
Vorgehen	Kalenay: $\frac{t_3 - t_0 - \Theta}{2}$	$Di = \sum A_j / n - A_i$
Schwächen	fesymetrische Kalenz	Zentraler Server Intern

VS - KVB - Ex 5

(5-3) Berkeley Algorithm

Die Uhren in einem VS mit 5 Rechnern ($R_i, i \in [0..4]$) werden durch Anwendung des Berkeley Algorithmus synchronisiert. Die Rechneruhren C_i haben zur Zeit T den Stand:

$C_0(T) = 1:01:07$ $\quad C_1(T) = 1:01:03$

$C_2(T) = 1:01:09$ $\quad C_3(T) = 1:01:04$

$C_4(T) = 1:01:07$

Master sei R_0, $RTT = 2$ sek aber ein Weg $\frac{RTT}{2}$

1) Wenn die Synchronisation zum Zeitpunkt T vom Master initiiert wird, welche Antworten A_i erhält er dann von den Slaves und welche Korrekturen D_i teilt er anschließend?

$A_0 = 0$ $\quad , D = D_0 = -1 = \leq 1_j / n$

$A_1 = A_1' - \frac{RTT}{2} = -4$ $\quad \rightarrow D_1 = D - A_1 = +3$

$A_1' = C_1(T) + \frac{RTT}{2} - C_0(T) = -3$

$A_2 = 2$ $\quad \rightarrow D_2 = -3$

$A_3 = -3$ $\quad \rightarrow D_3 = 2$

$A_4 = 0$ $\quad \rightarrow D_4 = +1$

VS-KVB-ExS

S-3

2) Angenommen R_2 verfügt neun Zeitfenster über die korrekte Uhrzeit ($C_2(T) = T$). Was würde durch die Synchronisation bewirkt?

$D_1 = 3 - 2 = 1$

$D_2 = 3 - 5 =$